```
from tensorflow.keras.datasets import imdb
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(
    num_words=10000)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
```

Import IMDB database from keras. Establish train data/labels and test data/labels. Limit data to to only the top 10,000 most frequent words for manageable vector data size

```
import numpy as np
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        for j in sequence:
            results[i, j] = 1.
    return results
x_train = vectorize_sequences(train_data)
x_test = vectorize_sequences(test_data)
y_train = np.asarray(train_labels).astype("float32")
y_test = np.asarray(test_labels).astype("float32")
```

lists in data were of different lengths. We need a tensor for the neural network to work. We do that by doing multi-hot coding, where we create a tensor of shape (25000, 10000) that contains ones and zeros corresponding to the position of the word index. We also convert the test and train labels into numpy arrays with 32-bit floating-point numbers

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
```

Implement three layers. Two use the rectified linear unit function which introduces non-linearity and expands our hypothesis space. Each has 16 neurons and each neuron produces a tensor with randomized weights plus a single randomized bias value. The third and final layer is uses a sigmoid activation method, which will output a single value between one and zero for each sample. This value is a probability.

```
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
```

Becuase we are doing a binary classification problem and our output is a probability, we choose binary crossentropy as our loss function. We use rmsprop as an optimizer. This adjust the learning rate adaptively based on the gradient size. Finally, we set our metric to accuracy.

```
x_val = x_train[:10000]
partial_x_train = x_train[10000:]
y_val = y_train[:10000]
partial_y_train = y_train[10000:]
```

We split the data. 15000 for training and 10000 for validation.

```
history = model.fit(partial_x_train,
                    partial_y_train,
                    epochs=20,
                    batch_size=512,
                    validation_data=(x_val, y_val))
```

```
Epoch 1/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 8s 221ms/step - accuracy: 0.8013 - loss: 0.5004 - val_accuracy: 0.8656 - val_loss: 0.3813
Epoch 2/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step - accuracy: 0.8959 - loss: 0.3099 - val_accuracy: 0.8797 - val_loss: 0.3159
Epoch 3/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.9235 - loss: 0.2331 - val_accuracy: 0.8897 - val_loss: 0.2822
Epoch 4/20
```

```
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.9366 - loss: 0.1912 - val_accuracy: 0.8814 - val_loss: 0.2953
Epoch 5/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9491 - loss: 0.1592 - val_accuracy: 0.8865 - val_loss: 0.2831
Epoch 6/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9541 - loss: 0.1397 - val_accuracy: 0.8736 - val_loss: 0.3190
Epoch 7/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - accuracy: 0.9638 - loss: 0.1171 - val_accuracy: 0.8846 - val_loss: 0.2952
Epoch 8/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9687 - loss: 0.1025 - val_accuracy: 0.8832 - val_loss: 0.3115
Epoch 9/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9746 - loss: 0.0889 - val_accuracy: 0.8797 - val_loss: 0.3262
Epoch 10/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9777 - loss: 0.0780 - val_accuracy: 0.8816 - val_loss: 0.3387
Epoch 11/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 50ms/step - accuracy: 0.9840 - loss: 0.0649 - val_accuracy: 0.8799 - val_loss: 0.3551
Epoch 12/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 55ms/step - accuracy: 0.9869 - loss: 0.0548 - val_accuracy: 0.8764 - val_loss: 0.3974
Epoch 13/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 36ms/step - accuracy: 0.9903 - loss: 0.0477 - val_accuracy: 0.8769 - val_loss: 0.3941
Epoch 14/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9921 - loss: 0.0396 - val_accuracy: 0.8683 - val_loss: 0.4367
Epoch 15/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9937 - loss: 0.0346 - val_accuracy: 0.8734 - val_loss: 0.4398
Epoch 16/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - accuracy: 0.9935 - loss: 0.0310 - val_accuracy: 0.8741 - val_loss: 0.4660
Epoch 17/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9957 - loss: 0.0254 - val_accuracy: 0.8723 - val_loss: 0.4811
Epoch 18/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9978 - loss: 0.0192 - val_accuracy: 0.8673 - val_loss: 0.5131
Epoch 19/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - accuracy: 0.9969 - loss: 0.0187 - val_accuracy: 0.8717 - val_loss: 0.5261
Epoch 20/20
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - accuracy: 0.9990 - loss: 0.0140 - val_accuracy: 0.8708 - val_loss: 0.5492
```

```
results = model.evaluate(x_test, y_test)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8597 - loss: 0.5862
```

Here we see the accuracy and loss for validation data across all 20 epochs. Finally we evaluate the model on the test data, achieving an accuracy of 86% and a loss of 0.586 binary cross-entropy units.

```
def build_model(units, activations, loss_fn, regularizer=None):
    model = keras.Sequential()

    # input layer with proper shape
    model.add(layers.Dense(units[0],
                           activation=activations[0],
                           kernel_regularizer=regularizer,
                           input_shape=(10000,)))

    # Add layers with differing units, activation methods, and regulizers based
    # on input
    for u, act in zip(units[1:-1], activations[1:-1]):
        model.add(layers.Dense(u,
                               activation=act,
                               kernel_regularizer=regularizer))

    # Final output layer
    model.add(layers.Dense(units[-1],
                           activation=activations[-1],
                           kernel_regularizer=regularizer))

    model.compile(
        optimizer="rmsprop",
        loss=loss_fn,
        metrics=["accuracy"]
    )
    return model
```

For adjusting the model's number of layers, number of units, activation method, and applying a weight regulizer, we will create a method that loops through a dictionary. This saves us from having to copy paste the model's code over and over for each configuration

```
experiments = [
    {
        "units": [16, 16, 1],
        "activations": ["relu", "relu", "sigmoid"],
```

```
            "loss": "binary_crossentropy",
            "regularizer": None
        },
        {
            "units": [16, 1],
            "activations": ["relu", "sigmoid"],
            "loss": "binary_crossentropy",
            "regularizer": None
        },
        {
            "units": [32, 32, 1],
            "activations": ["relu", "relu", "sigmoid"],
            "loss": "binary_crossentropy",
            "regularizer": None
        },
        {
            "units": [64, 64, 1],
            "activations": ["relu", "relu", "sigmoid"],
            "loss": "binary_crossentropy",
            "regularizer": None
        },
        {
            "units": [64, 128, 1],
            "activations": ["relu", "relu", "sigmoid"],
            "loss": "binary_crossentropy",
            "regularizer": None
        },
        {
            "units": [16, 16, 1],
            "activations": ["relu", "relu", "sigmoid"],
            "loss": "mse",
            "regularizer": None
        },
        {
            "units": [16, 16, 1],
            "activations": ["tanh", "tanh", "sigmoid"],
            "loss": "mse",
            "regularizer": None
        },
        {
            "units": [16, 16, 1],
            "activations": ["relu", "relu", "sigmoid"],
            "loss": "binary_crossentropy",
            "regularizer": keras.regularizers.l2(0.002)
        }
    ]
```

The first experiment is our baseline from the textbook IMDB example. The second experiment changes the number of layers to 2 (1 hidden and 1 output). The 3rd, 4th, and 5th expirements adjust the number of units in the various layers.The 6th experiment adjusts the loss function to "mse". The 7th experiment implements a weight regulizer, which penalizes large weights to avoid overfitting.

```
    for exp in experiments:
        model = build_model(
            units=exp["units"],
            activations=exp["activations"],
            loss_fn=exp["loss"],
            regularizer=exp["regularizer"]
        )

        history = model.fit(
            x_train,
            y_train,
            epochs=4,
            batch_size=512,
            validation_split=0.2,
            verbose=0
        )

        val_acc = history.history["val_accuracy"][-1]

        test_loss, test_acc = model.evaluate(
            x_test,
            y_test,
            verbose=0
        )
```

```
        results.append({
            "units": exp["units"],
            "activations": exp["activations"],
            "loss": exp["loss"],
            "regularizer": exp["regularizer"] is not None,
            "val_acc": val_acc,
            "test_acc": test_acc
        })
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Here we iterate through our experiment library, building each model to the specifications of the library item, then adding the validation and test accuracy to the results list. We are using 4 epochs here to save computation power across the many models (and because the text book identified this as the point where overfitting becomes an issue). We are also using the built-in keras validation_split argument to separate training and validation data.

```
import pandas as pd
df = pd.DataFrame(results)
pd.set_option("display.max_rows", None)
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", None)

df
```

1 to 10 of 10 entries Filter

index ▼	0
9	{'units': [16, 16, 1], 'activations': ['relu', 'relu', 'sigmoid'], 'loss': 'binary_crossentropy', 'regularizer': True, 'val_acc': 0.8930000066757202, 'test_acc': 0.8863199949264526}
8	{'units': [16, 16, 1], 'activations': ['tanh', 'tanh', 'sigmoid'], 'loss': 'mse', 'regularizer': False, 'val_acc': 0.8862000107765198, 'test_acc': 0.878000020980835}
7	{'units': [16, 16, 1], 'activations': ['relu', 'relu', 'sigmoid'], 'loss': 'mse', 'regularizer': False, 'val_acc': 0.878600001335144, 'test_acc': 0.8712400197982788}
6	{'units': [64, 128, 1], 'activations': ['relu', 'relu', 'sigmoid'], 'loss': 'binary_crossentropy', 'regularizer': False, 'val_acc': 0.8830000162124634, 'test_acc': 0.8787999749183655}
5	{'units': [64, 64, 1], 'activations': ['relu', 'relu', 'sigmoid'], 'loss': 'binary_crossentropy', 'regularizer': False, 'val_acc': 0.8813999891281128, 'test_acc': 0.8692799806594849}
4	{'units': [32, 32, 1], 'activations': ['relu', 'relu', 'sigmoid'], 'loss': 'binary_crossentropy', 'regularizer': False, 'val_acc': 0.8881999850273132, 'test_acc': 0.8821600079536438}
3	{'units': [16, 1], 'activations': ['relu', 'sigmoid'], 'loss': 'binary_crossentropy', 'regularizer': False, 'val_acc': 0.8944000005722046, 'test_acc': 0.8885200023651123}
2	{'units': [16, 16, 1], 'activations': ['relu', 'relu', 'sigmoid'], 'loss': 'binary_crossentropy', 'regularizer': False, 'val_acc': 0.8888000249862671, 'test_acc': 0.8845599889755249}

Generate code with df New interactive sheet

Double-click (or enter) to edit

We display the parameters of each experiment, the validation accuracy, and test accuracy for each. The two models with the highest test accuracy were experiment #2 (only 1 hidden layer) with a test accuracy of 89% and experiment #8 (weight regulizer) with also with a test accuracy of 89%. As a last experiment, lets combine these two design choices to see if we can get the accuracy even higher

```
model = keras.Sequential([
    layers.Dense(
        16,
        activation="relu",
        kernel_regularizer=keras.regularizers.l2(0.001)
    ),
    layers.Dense(
        1,
        activation="sigmoid",
    )
])

model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])

final_history = model.fit(
    x_train,
    y_train,
```

```
    epochs=4,
    batch_size=512,
    validation_split=0.2,
    verbose=0
)
final_test = model.evaluate(x_test, y_test)
print(final_test)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8807 - loss: 0.3404
[0.34040725231170654, 0.8807200193405151]
```

After trying a combination of layer numbers, neuron (input) numbers, loss functions, activation methods, and applying weight regulation, we have trouble achieving accuracy above 88-89%. Continuing to tune the dense layers is therefore probably not the best way to get more accuracy out of this particular model. To increase accuracy further, my next step would be to start looking at the way the input is represented, potentially implementing things like term frequency scores.